

JENKINS & DOCKER PIPELINES

Day 15: Multi-Stage Docker Builds & Flask App CI/CD Orchestration

1. Multi-Stage Dockerfile Analysis

Multi-stage Docker builds allow developer pipelines to separate the compilation/build environment from the final execution runtime. This approach dramatically reduces the size of the production container image and enhances security by excluding build tools, development dependencies, and raw source code from the final runtime image.

```
# ===== Build Stage =====
FROM python:3.9-slim AS build
WORKDIR /app
COPY app.py .
RUN pip install --no-cache-dir flask

# ===== Runtime Stage =====
FROM python:3.9-slim
WORKDIR /app
# Copy installed packages from build stage
COPY --from=build /usr/local/lib/python3.9 /usr/local/lib/python3.9
# Copy application files from build stage
COPY --from=build /app /app
EXPOSE 5000
ENV PYTHONUNBUFFERED=1
CMD ["python", "app.py"]
```

Dockerfile Instruction Specification:

- **FROM ... AS build:** Initializes the build container using a Python base. It downloads the SDK needed for package compilation.
- **RUN pip install ...:** Compiles and installs the Flask library inside the build stage image without storing temporary pip caches.
- **COPY --from=build:** Crucial command. Copies ONLY compile-time site-packages and source code from the build image to the clean runtime stage, keeping production images lightweight (no pip, headers, or compilers).
- **ENV PYTHONUNBUFFERED=1:** Forces Python standard stdout/stderr streams to be unbuffered, ensuring real-time printing of application logs to stdout.
- **CMD ["python", "app.py"]:** Sets the default command to start the Flask application when the container is executed.

2. Jenkins CI/CD Pipeline Orchestration

The Jenkins pipeline automates checkout, building, testing, publishing, and running the application container. It defines dynamic global variables in the "environment" block: `IMAGE_NAME` controls the repository coordinate, and `IMAGE_TAG` is dynamically tagged with the unique build number (`${BUILD_NUMBER}`) to guarantee artifact immutability.

Pipeline Stages 1 to 3 Specification:

Stage 1: Checkout Code

Command: `checkout scm`

Pulls the source code of the project (containing the Dockerfile, `app.py`, and dependencies) from the Git repository associated with the Jenkins job, using configuration defined by the Source Control Management settings.

Stage 2: Build Docker Image

Command: `docker build -t $IMAGE_NAME:$IMAGE_TAG .`

Compiles the multi-stage Dockerfile in the workspace. It creates a local image and tags it with the repository username, application name, and the current build number.

Stage 3: Test Container

Command: `docker run -d -> sleep 10 -> docker ps -> docker stop & rm`

Verifies the integrity of the built image by running it as a detached container on port 5000. It sleeps for 10 seconds to allow the app to initialize, runs "docker ps" to verify the container remains healthy (not crashed), and cleans up the test container.

Pipeline Stages 4 to 6 & Post-Actions Specification:

Stage 4: Docker Login

Command: `docker login -u "$DOCKER_USER" --password-stdin`

Secures credentials with Jenkins' "withCredentials" helper. It binds username and password configurations to environment variables, allowing the pipeline to authenticate with DockerHub securely without printing secrets to logs.

Stage 5: Push Docker Image

Command: `docker push $IMAGE_NAME:$IMAGE_TAG / latest`

Pushes the specific build-numbered image tag to DockerHub. It also tags and pushes a secondary "latest" reference, ensuring downstream deployments have access to both specific revisions and current versions.

Stage 6: Deploy Application

Command: `docker run -d --name flask-app -p 5000:5000 ...`

Stops and deletes any previously running production container instance, then launches the newly compiled image as a daemon to complete the continuous deployment process.

Post-Actions: success / failure / always

Command: `docker image prune -f`

Runs post-execution instructions. Prints log messages based on success or failure state. The "always" block executes "docker image prune" to delete intermediate build stages and cache layers, saving server disk space.

3. Jenkinsfile Pipeline Source Code

```
pipeline {
  agent any
  environment {
    IMAGE_NAME = "YOUR_DOCKERHUB_USERNAME/flask-hospital-app"
    IMAGE_TAG = "${BUILD_NUMBER}"
  }
  stages {
    stage('Checkout Code') { steps { checkout scm } }
    stage('Build Docker Image') { steps { sh 'docker build -t $IMAGE_NAME:$IMAGE_TAG .' } }
    stage('Test Container') {
      steps { sh 'docker run -d --name test-container -p 5000:5000 $IMAGE_NAME:$IMAGE_TAG; sleep 10;
docker ps; docker stop test-container; docker rm test-container' }
    }
    stage('Docker Login') {
      steps { withCredentials([usernamePassword(credentialsId: 'dockerhub-cred', usernameVariable:
'DOCKER_USER', passwordVariable: 'DOCKER_PASS')]) { sh 'echo "$DOCKER_PASS" | docker login -u "$DOCKER_USER"
--password-stdin' } }
    }
    stage('Push Docker Image') { steps { sh 'docker push $IMAGE_NAME:$IMAGE_TAG; docker tag $IMAGE_NAME:
$IMAGE_TAG $IMAGE_NAME:latest; docker push $IMAGE_NAME:latest' } }
    stage('Deploy Application') { steps { sh 'docker run -d --name flask-app -p 5000:5000 $IMAGE_NAME:$IMAGE_TAG' } }
    post { always { sh 'docker image prune -f || true' } }
  }
}
```